

Portafolio de Evidencias de Aprendizaje — Segundo Corte

Preparatorio para EV-SUM-02 (Semana 16)

Carlos José Carpio Mendoza — Modalidad individual — Carácter formativo

Fecha de elaboración: 12 de agosto de 2026

Este portafolio recoge, de forma individual, el aprendizaje del segundo corte de Aplicaciones Distribuidas dentro del PFC de equipo (sistema de soporte técnico ISP con tres nodos CockroachDB y un pipeline de procesamiento de tickets en Spark/pandas). Cubre cuatro puntos: evidencias concretas de código y análisis propios, la contribución individual documentada en el repositorio, la autoevaluación de logro sobre los RA de las Unidades 2 y 4, y las preguntas técnicas que aún no domino con seguridad.

1. Evidencias concretas de aprendizaje

Las tres evidencias siguientes provienen del trabajo individual realizado en los módulos de *cluster distribuido CockroachDB* (fragmentación y tolerancia a fallos) y *pipeline de procesamiento Spark/pandas* (verificación de equivalencia y modelos de rendimiento).

1.1 Evidencia 1 — Fragmentación y tolerancia a fallos en un cluster distribuido

Al reparticionar la tabla `tickets` de fragmentación por zona geográfica a fragmentación por `fecha_apertura` (ADR-0003), tuve que ajustar tanto el esquema de particiones como la política de replicación:

```
-- init_db.sql: particionado trimestral por fecha_apertura (created_at)
PARTITION BY RANGE (created_at) (
  PARTITION q1_2026 VALUES FROM (MINVALUE) TO ('2026-04-01'),
  PARTITION q2_2026 VALUES FROM ('2026-04-01') TO ('2026-07-01'),
  PARTITION q3_2026 VALUES FROM ('2026-07-01') TO ('2026-10-01'),
  PARTITION q4_2026 VALUES FROM ('2026-10-01') TO (MAXVALUE)
);

-- zones.sql: replicacion uniforme (quorum de escritura = 2 de 3) porque una
-- particion trimestral por fecha_apertura mezcla tickets de las 3 zonas
ALTER TABLE tickets CONFIGURE ZONE USING num_replicas = 3;
```

Como ya no existe correspondencia 1:1 entre partición y localidad de nodo (una partición trimestral mezcla tickets de las tres zonas), el factor de replicación ya no se ancla por `constraint` de localidad: se exige de forma uniforme sobre toda la tabla, para tolerar la caída de cualquier nodo sin perder disponibilidad de escritura.

También resolví un problema real de estabilidad del cluster: en desarrollo local (Docker sobre WSL2) el reloj de la VM se desincroniza tras cada suspensión del host, y CockroachDB rechaza arrancar si el desfase supera el `max-offset` por defecto (500 ms):

```
# docker-compose.cockroach.yml
command: start --insecure --max-offset=4500ms --locality=zone=quevedo-centro --join=roach1,roach2,roach3
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/health?ready=1"]
```

Análisis: este ajuste no es aplicable a un cluster real (solo compensa un artefacto del entorno de desarrollo), lo cual me obligó a documentar explícitamente por qué existe, para no confundirlo con una decisión de diseño de producción.

1.2 Evidencia 2 — Verificación de equivalencia entre motor secuencial y motor distribuido

Para las cinco transformaciones exigidas, comparé la salida de pandas (secuencial) contra PySpark (distribuido) *sin* comparar fila a fila, porque un motor distribuido no garantiza el orden de salida salvo que se ordene explícitamente:

```
def verificar_t2(pandas_dir, spark_dir):
    p, s = cargar("T2_agrupacion", pandas_dir, spark_dir)
    p = p.sort_values("state").reset_index(drop=True)
    s = s.sort_values("state").reset_index(drop=True)
    return {
        "cardinalidad_igual": len(p) == len(s),
        "suma_igual": int(p["total_tickets"].sum()) == int(s["total_tickets"].sum()),
    }
```

Análisis: verificar por cardinalidad y agregados de control, en vez de igualdad estricta, fue una decisión de diseño y no un atajo: refleja una propiedad real de los sistemas distribuidos (no determinismo del orden) que no existe en pandas.

1.3 Evidencia 3 — Reflexión sobre el ajuste de la Ley de Amdahl con datos propios

Con los tiempos medidos (mediana de 5 corridas) para $N = 1, 2, 4$ executors, ajusté la fracción paralela p (forma de Karp–Flatt) y obtuve:

```
p_ajustado (min. cuadrados) = 0.460 S_max = 1.85 N para 90% de S_max = 7.7
Speedup observado: N=1 -> 1.00 | N=2 -> 0.98 | N=4 -> 1.68
```

Reflexión: el resultado con $N = 2$ (speedup < 1) fue el hallazgo más formativo del corte: al inicio esperaba que más executors siempre redujera el tiempo, pero el overhead de coordinación (planificación de tareas, shuffle) superó la ganancia con un dataset y una fracción paralela todavía moderados. Entendí en la práctica –y no solo en la fórmula– la diferencia entre paralelismo teórico y paralelismo efectivo, y por qué $S_{max} = 1,85$ es un techo real impuesto por la parte serial del pipeline, no un problema de configuración.

2. Contribución individual al PFC (documentada con commits de GitHub)

Nota metodológica: en la fase de integración del equipo el orden de subida de varios commits fue normalizado, por lo que aquí documento la contribución **por el código y los módulos entregados**, usando el hash y mensaje de commit como referencia de trazabilidad en el repositorio, no como prueba de la línea de tiempo exacta de trabajo.

Commit	Módulo	Aporte
72a6539	db-cluster	Arquitectura inicial del cluster CockroachDB de 3 nodos.
0083fca	db-cluster	Estabilización local: desfase de reloj, puertos, healthchecks.
2b0e134	db-cluster	Reparticionamiento por <code>fecha_apertura</code> y ajuste de memoria de nodos.
342718e	report-service	Microservicio de reportes con modelo de lectura CQRS.
6d50f1e	frontend	Dashboard de tickets, panel admin y vista de reportes.
5956b4e	devops	Script único de arranque de los 5 microservicios.
f6e133a	spark/pandas	Verificación de equivalencia entre motores.
8ce0da7	spark/pandas	Protocolo de benchmark (1+5 corridas, medianas).
c5fd274	análisis	Figuras a 300 DPI y ajuste de la Ley de Amdahl.

3. Autoevaluación del nivel de logro — RA Unidades 2 y 4

Resultado de aprendizaje	Nivel	Justificación
U2. Diseña arquitecturas de datos distribuidas aplicando particionamiento y replicación con tolerancia a fallos.	Logrado	Implementé el reparticionamiento de <code>tickets</code> por <code>fecha_apertura</code> , configuré replicación uniforme (<code>num_replicas=3</code>) y diagnosticué/resolví un fallo real de arranque por desfase de reloj entre nodos (Evidencia 1).
U4. Evalúa rendimiento y escalabilidad de procesamiento distribuido con modelos analíticos (Amdahl/Gustafson-Barsis) y protocolos experimentales.	En proceso	Apliqué correctamente el modelo y el protocolo de medición (Evidencias 2 y 3), pero el resultado con $N=2$ me mostró que aún no tengo intuición sólida para <i>predecir</i> cuándo el overhead de coordinación domina sobre la ganancia paralela antes de medir; sé interpretar el dato después de obtenerlo, no anticiparlo con seguridad.

(Redacción de RA propuesta a partir del contenido real trabajado en el corte; verificar contra el texto exacto de la guía de aprendizaje del curso antes de la entrega final.)

4. Preguntas técnicas que más me cuesta responder

1. **Elección de la clave de fragmentación (shard key).** Tuvimos que migrar de fragmentar `tickets` por zona geográfica a fragmentarla por `fecha_apertura`. Entiendo el síntoma (hotspots) pero no tengo un método sistemático para *anticipar*, antes de implementar, si una clave candidata va a producir puntos calientes en un cluster distribuido.
2. **Overhead de coordinación vs. paralelismo real.** Con $N=2$ executors obtuve $\text{speedup} < 1$ frente a $N=1$. Puedo explicar el resultado *a posteriori* (shuffle, planificación de tareas), pero no sé calcular de antemano, para un pipeline y un tamaño de dato dados, a partir de qué N el overhead deja de dominar.
3. **Trade-off CAP en la práctica.** Configuré `num_replicas=3` (quorum de escritura 2 de 3) por indicación de la guía, pero no tengo claro cómo decidir yo mismo, para un caso de uso distinto, qué factor de replicación y qué nivel de consistencia elegir sin sobre-dimensionar el cluster ni sacrificar disponibilidad ante la caída de un nodo.

Conclusión personal

Cerrando el segundo corte, el patrón común entre el cluster CockroachDB y el pipeline Spark es que ambos me exigieron pasar de *aplicar* una fórmula o una configuración recomendada a *justificar* por qué esa configuración es correcta para el caso concreto (una clave de fragmentación, un factor de replicación, un número de executors). Las tres preguntas de la sección 4 marcan exactamente ese límite entre lo que ya puedo ejecutar y verificar, y lo que todavía no puedo predecir con confianza antes de medir. Ese es el foco con el que quiero llegar a EV-SUM-02.